

Doodle Jump mit Pygame

Luca Voithofer

February 20, 2025

1 Game Over und Restart-Funktion

Bisher hatte unser Spiel keine Möglichkeit, nach einem Game Over neu zu starten. Dafür fügen wir nun eine Restart-Funktion hinzu, die das Spiel zurücksetzt, wenn die Leertaste (SPACE) gedrückt wird.

1.1 Game Over Status hinzufügen

Zunächst benötigen wir eine Variable, die speichert, ob das Spiel vorbei ist.

```
1 # Game Over Text
2 gameOverFont = pygame.font.Font("freesansbold.ttf", 64)
3 gameOverState = False
```

- **gameOverState:** Speichert, ob das Spiel vorbei ist (True bedeutet Game Over).

1.2 Game Reset-Funktion

Damit das Spiel von vorne beginnt, setzen wir alle Variablen auf ihre Anfangswerte zurück.

```
1 #Reset Game
2 def resetGame():
3     global playerX, playerY, changeX, enemyX, enemyY,
4         changeXenemy, changeYenemy
5     global NumberOfEnemies, bulletX, bulletY, changeXbullet,
6         changeYbullet
7     global bulletState, gameOverState, score
8
9     # Reset playerposition
10    playerX = 370
11    playerY = 480
```

```

10     changeX = 0
11
12     # Replace enemies
13     enemyX = []
14     enemyY = []
15     changeXenemy = []
16     changeYenemy = []
17     NumberOfEnemies = 8
18
19     for i in range(NumberOfEnemies):
20         enemyX.append(random.randint(0,736))
21         enemyY.append(random.randint(50,150))
22         changeXenemy.append(0.3)
23         changeYenemy.append(40)
24
25     # Reset gameOverState
26     gameOverState = False
27
28     # Reset bullet
29     bulletX = 0
30     bulletY = 480
31     changeXbullet = 0
32     changeYbullet = 0.5
33     bulletState = "ready"
34
35     # Reset score
36     score = 0

```

- **Alle Spielvariablen** werden auf ihre Ausgangswerte zurückgesetzt.
- **Gegner** werden neu platziert diesmal mit 8 statt 6 Aliens.
- **gameOverState** wird wieder auf False gesetzt, sodass das Spiel neu startet.
- **Score und Kugelzustand** werden zurückgesetzt, damit die Punkte von null beginnen.

1.3 Restart einleiten

Nun erlauben wir dem Spieler, das Spiel durch Drücken von SPACE neuzustarten, wenn sich das Spiel im GameState befindet.

```

1 # Neustart nach Game Over
2 if event.key == pygame.K_SPACE and gameOverState == True:
3     resetGame()

```

2 Highscore hinzufügen

Ein Highscore-System speichert die höchste erreichte Punktzahl und zeigt sie an. Dazu fügen wir eine **Highscore-Variable**, eine **Anzeige-Funktion** und die **Aktualisierung der Highscore-Werte** hinzu.

2.1 Highscore-Variable

Zunächst definieren wir die Variable `highscore`, die den höchsten erreichten Punktestand speichert.

```
1 # Highscore-Variable
2 highscore = 0
```

- **highscore = 0:** Initialisiert die Highscore-Variable mit 0.

2.2 Highscore-Anzeige

Nun erstellen wir eine Funktion, die den Highscore auf dem Bildschirm anzeigt.

```
1 # Highscore anzeigen
2 def showHighScore(x, y):
3     scorePicture = font.render("HighScore: " + str(highscore)
4                               , True, (255,255,255))
5     screen.blit(scorePicture, (x, y))
```

- **font.render(...):** Erstellt ein Textbild für den Highscore.
- **screen.blit(...):** Zeichnet den Highscore-Text auf den Bildschirm.
- **x, y:** Definieren die Position der Anzeige.

2.3 Highscore in der Game Loop aktualisieren

Die Highscore-Variable muss aktualisiert werden, wenn der aktuelle Score höher ist als der bisherige Highscore.

```
1 # Highscore aktualisieren
2 if score > highscore:
3     highscore = score
```

- Falls der aktuelle Punktestand (`score`) größer als der gespeicherte `highscore` ist, wird der Highscore aktualisiert.

2.4 Highscore auf dem Bildschirm anzeigen

Nachdem der Highscore aktualisiert wurde, muss die Anzeige-Funktion aufgerufen werden.

```
1 # Highscore anzeigen
2 showHighScore(10, 45)
```

- **showHighScore(10, 45):** es wird die showHighScore-Funktion aufgerufen und der Highscore wird auf die Koordinaten (10, 45) gesetzt